

LLM 常见问题解答

本文档整理了关于大语言模型（LLM）和相关技术的常见问题及详细解答。

1. 损失函数是什么？有哪些常见的损失函数？

什么是损失函数？

损失函数（Loss Function）是机器学习中用来衡量模型预测值与真实值之间差异的函数。在训练过程中，模型通过最小化损失函数来学习参数，使预测结果尽可能接近真实值。

常见的损失函数

1. 交叉熵损失（Cross-Entropy Loss）

- 用途：主要用于分类任务，特别是多分类问题
- 公式： $L = - \sum_{i=1}^n y_i \log(\hat{y}_i)$
- 特点：在LLM训练中广泛使用，特别是在语言建模和分类任务中
- 代码示例：

```
loss = torch.nn.functional.cross_entropy(logits, target_batch)
```

2. 均方误差（Mean Squared Error, MSE）

- 用途：主要用于回归任务
- 公式： $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

3. 平均绝对误差（Mean Absolute Error, MAE）

- 用途：回归任务
- 公式： $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$

4. 二元交叉熵（Binary Cross-Entropy）

- 用途：二分类任务
- 公式： $BCE = - \frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$

5. Focal Loss

- 用途：处理类别不平衡问题
- 特点：通过降低易分类样本的权重，使模型更关注难分类样本

6. KL散度（Kullback-Leibler Divergence）

- 用途：衡量两个概率分布的差异，常用于模型蒸馏和对比学习

2. BERT模型和GPT模型的区别

架构差异

特性	BERT	GPT
架构类型	Encoder（编码器）	Decoder（解码器）
训练方式	双向上下文	单向上下文（自回归）
注意力机制	双向自注意力	因果自注意力（带掩码）
主要任务	理解任务（分类、NER、QA）	生成任务（文本生成、对话）

详细说明

BERT（Bidirectional Encoder Representations from Transformers）

- 核心特点：
 - 使用Transformer的Encoder部分
 - 双向理解上下文，可以同时看到前后文信息
 - 通过掩码语言模型（MLM）和下一句预测（NSP）任务进行预训练
- 适用场景：
 - 文本分类
 - 命名实体识别（NER）
 - 问答系统（QA）
 - 情感分析
- 局限性：
 - 不适合文本生成任务
 - 无法进行自回归生成

GPT (Generative Pre-trained Transformer)

- **核心特点：**
 - 使用Transformer的Decoder部分
 - 单向上下文，只能看到当前位置之前的信息
 - 通过自回归语言建模进行预训练
- **适用场景：**
 - 文本生成
 - 对话系统
 - 代码生成
 - 创意写作
- **优势：**
 - 强大的生成能力
 - 可以处理长文本序列
 - 支持零样本和少样本学习

总结

- **BERT：**擅长理解和分析任务，是"阅读理解"专家
 - **GPT：**擅长生成和创作任务，是"文本创作"专家
-

3. 什么是自注意力？什么是多头注意力？

自注意力 (Self-Attention)

自注意力机制是Transformer架构的核心，允许模型在处理序列时，每个位置都能关注到序列中的所有其他位置。

工作原理

1. **输入：**序列中的每个token被转换为embedding向量
2. **计算注意力分数：**通过Query、Key、Value矩阵计算每个token对其他token的关注度
3. **加权求和：**根据注意力权重对Value进行加权求和，得到上下文向量

简化版自注意力

```
# 输入矩阵 X: [seq_len, embed_dim]
attn_scores = X @ X.T # [seq_len, seq_len]
attn_weights = softmax(attn_scores, dim=-1)
contexts = attn_weights @ X # [seq_len, embed_dim]
```

完整版自注意力（带Q、K、V）

```
# 计算Q、K、V
Q = X @ W_Q # Query矩阵
K = X @ W_K # Key矩阵
V = X @ W_V # Value矩阵

# 计算注意力分数
d_K = K.shape[-1]
attn_scores = Q @ K.T / sqrt(d_K) # 缩放点积注意力
attn_weights = softmax(attn_scores, dim=-1)

# 计算上下文向量
contexts = attn_weights @ V
```

多头注意力（Multi-Head Attention）

多头注意力是对自注意力的扩展，通过并行运行多个注意力"头"来捕获不同类型的信息。

核心思想

- **多个注意力头**：每个头学习不同的表示子空间
- **并行计算**：所有头同时计算，提高效率
- **信息融合**：将多个头的输出拼接后通过线性层融合

工作流程

1. **分割**：将Q、K、V分割成多个头（例如8个头）
2. **并行计算**：每个头独立计算自注意力
3. **拼接**：将所有头的输出拼接
4. **线性变换**：通过输出投影层得到最终结果

优势

- **捕获不同类型的关系**：语法、语义、长距离依赖等
- **提高模型表达能力**：不同头关注不同的特征

- 增强泛化能力：通过多视角理解文本

公式

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{其中每个头: } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

4. Transformer中 Q、K、V 三个矩阵有什么用？

基本概念

Q (Query)、K (Key)、V (Value) 是注意力机制中的三个核心矩阵，它们共同实现了"查询-键-值"的注意力计算模式。

各矩阵的作用

Query (Q) - 查询矩阵

- 作用：表示"我想知道什么"
- 含义：当前token想要查询的信息
- 类比：就像在图书馆中，你提出的问题（查询）

Key (K) - 键矩阵

- 作用：表示"我有什么信息"
- 含义：每个token提供的可匹配信息
- 类比：就像图书馆中每本书的索引标签（键）

Value (V) - 值矩阵

- 作用：表示"实际的信息内容"
- 含义：每个token实际包含的信息内容
- 类比：就像图书馆中每本书的实际内容（值）

工作流程

1. 匹配阶段：Q与K进行点积运算，计算相似度分数

```
attn_scores = Q @ K.T
```

- 分数越高，表示该位置的信息与当前查询越相关

2. 归一化阶段：通过softmax将分数转换为权重

```
attn_weights = softmax(attn_scores / sqrt(d_K))
```

3. 聚合阶段：用权重对V进行加权求和

```
contexts = attn_weights @ V
```

为什么需要三个矩阵？

- **灵活性**：Q、K、V可以学习不同的表示，使模型更灵活
- **表达能力**：分离查询、匹配、内容三个步骤，增强模型表达能力
- **可学习性**：通过训练，模型可以学习到最优的Q、K、V表示

实际例子

假设处理句子："The cat sat on the mat"

- **Query ("cat")**：查询"cat"相关的信息
- **Key ("sat")**：提供"sat"这个动作信息
- **Value ("sat")**：实际包含的语义信息（坐的动作）

通过Q·K计算，"cat"会发现"sat"与自己高度相关，因此会更多地关注"sat"的Value信息。

5. 训练和微调有什么区别？

训练（Training / Pretraining）

定义

训练（预训练）是指从零开始训练一个模型，使用大规模无标注数据进行自监督学习。

特点

- **数据规模**：通常使用TB级别的海量数据
- **计算资源**：需要大量GPU和长时间训练（数周甚至数月）
- **目标**：学习通用的语言表示和知识
- **任务类型**：无监督或自监督任务
 - 语言建模（预测下一个词）
 - 掩码语言模型（BERT）
 - 自回归生成（GPT）

示例

```
# 预训练：在大规模文本上学习语言模型
# 目标：最大化  $P(w_t | w_1, \dots, w_{t-1})$ 
loss = cross_entropy(model(input_ids), target_ids)
```

微调（Fine-tuning）

定义

微调是在预训练模型的基础上，使用特定任务的小规模标注数据进行进一步训练。

特点

- **数据规模**：通常使用较小规模的标注数据（几千到几万样本）
- **计算资源**：相对较少，可以在单GPU上完成
- **目标**：适应特定任务或领域
- **训练方式**：
 - **全量微调**：更新所有参数
 - **参数高效微调**：只更新部分参数（如LoRA、Adapter）

微调类型

1. 任务微调（Task-specific Fine-tuning）

- 针对特定任务（如分类、NER）进行微调
- 示例：在GPT模型上添加分类头，进行情感分析

2. 指令微调（Instruction Fine-tuning）

- 使用指令-回答对训练模型遵循指令
- 示例：使用Alpaca数据集训练模型理解指令

3. 领域微调 (Domain Fine-tuning)

- 在特定领域数据上继续预训练
- 示例：在医学文本上继续训练通用模型

代码示例

```
# 微调：冻结大部分参数，只训练最后几层
for param in model.parameters():
    param.requires_grad = False

# 只训练最后一层
for param in model.trf_blocks[-1].parameters():
    param.requires_grad = True

# 添加任务特定的输出层
model.out_head = nn.Linear(emb_dim, num_classes)
```

对比总结

特性	训练 (预训练)	微调
起点	随机初始化	预训练模型
数据	大规模无标注数据	小规模标注数据
时间	数周/月	数小时/天
资源	大量GPU集群	单GPU/少量GPU
目标	学习通用表示	适应特定任务
成本	极高	相对较低

为什么需要微调?

1. 任务适配：预训练模型是通用的，需要针对具体任务优化
2. 领域适应：在特定领域（如医学、法律）上表现更好
3. 效率：比从头训练快得多，成本低
4. 效果：通常比从头训练效果更好

6. Embedding有什么作用？有哪些Embedding模型？

Embedding的作用

Embedding（嵌入）是将离散的符号（如单词、字符）转换为连续的向量表示，使计算机能够理解 and 处理文本。

主要作用

1. 语义表示

- 将文本转换为数值向量
- 语义相似的词在向量空间中距离更近

2. 降维

- 将高维的one-hot编码转换为低维稠密向量
- 例如：词汇表10000维 → 512维向量

3. 特征提取

- 捕获词汇的语义、语法、上下文信息
- 为下游任务提供有用的特征

4. 可计算性

- 向量可以进行数学运算（加法、减法、点积）
- 支持相似度计算、聚类等操作

Embedding的类型

1. 词嵌入（Word Embedding）

- **Word2Vec**：CBOW和Skip-gram模型
- **GloVe**：基于全局词共现矩阵
- **FastText**：考虑子词信息

2. 上下文嵌入（Contextual Embedding）

- **ELMo**：双向LSTM，考虑上下文
- **BERT**：双向Transformer编码器
- **GPT**：单向Transformer解码器

3. 句子/文档嵌入（Sentence/Document Embedding）

- **Universal Sentence Encoder**
- **Sentence-BERT (SBERT)**
- **Instructor**

4. 多模态嵌入

- **CLIP**: 图像-文本联合嵌入
- **Audio Embeddings**: 音频嵌入
- **Video Embeddings**: 视频嵌入

LLM中的Embedding

在LLM中，Embedding通常包括：

1. Token Embedding

- 将token ID转换为向量
- 每个token对应一个可学习的向量

2. Position Embedding

- 编码token在序列中的位置信息
- 可以是可学习的或使用正弦位置编码

3. Segment Embedding (BERT等)

- 区分不同的句子或段落

特点

- **自训练**: LLM通常自己训练embedding层，不使用预训练的embedding
- **整体性**: Transformer和embedding构成一个整体
- **任务相关**: 不同LLM的embedding可能不同，针对特定任务优化

生成向量的步骤

长文本

- 分词 (单个词)
- Token化处理
- 向量 (Embedding)

常见Embedding模型

模型	类型	特点
Word2Vec	词嵌入	静态，不考虑上下文
GloVe	词嵌入	基于全局统计
ELMo	上下文嵌入	双向LSTM
BERT	上下文嵌入	双向Transformer
GPT	上下文嵌入	单向Transformer
Sentence-BERT	句子嵌入	专门用于句子相似度
OpenAI Embeddings	通用嵌入	text-embedding-ada-002等

7. Token的含义是什么？

Token的定义

Token是文本处理中的基本单位，是将文本分割后得到的最小可处理单元。

Token的类型

1. 词级Token (Word-level)

- 以单词为单位进行分割
- 示例: "Hello world" → ["Hello", "world"]
- 优点: 直观，易于理解
- 缺点: 词汇表大，无法处理未登录词

2. 字符级Token (Character-level)

- 以字符为单位进行分割
- 示例: "Hello" → ["H", "e", "l", "l", "o"]
- 优点: 词汇表小，可以处理任意文本
- 缺点: 序列长，难以捕获语义

3. 子词级Token (Subword-level) ★ 最常用

- 将单词分割成更小的子词单元

- 示例: "unhappiness" → ["un", "happiness"] 或 ["un", "happy", "ness"]
- 优点:
 - 平衡词汇表大小和语义表示
 - 可以处理未登录词
 - 捕获词根、前缀、后缀等语言结构

常见的Token化方法

1. BPE (Byte Pair Encoding)

- 原理: 迭代合并最频繁的字符对
- 使用: GPT系列、RoBERTa
- 示例:

```
初始: ['l', 'o', 'w', 'e', 'r', '</w>', 'n', 'e', 'w', 'e', 's', 't', '</w>']  
合并 'e' 's' → 'es'  
合并 'es' 't' → 'est'  
最终: ['low', 'er', '</w>', 'new', 'est', '</w>']
```

2. WordPiece

- 原理: 基于最大似然估计选择子词单元
- 使用: BERT
- 特点: 以##开头表示子词 (非词首)

3. SentencePiece

- 原理: 将文本和token化统一处理
- 使用: T5、ALBERT
- 特点: 可以处理多语言, 不需要预处理

4. Unigram Language Model

- 原理: 从大词汇表开始, 逐步删除不重要的子词
- 使用: 部分多语言模型

Token化的过程

原始文本: "Hello, how are you?"
↓
分词: ["Hello", ",", "how", "are", "you", "?"]
↓
Token化: ["Hello", ",", "how", "are", "you", "?"]
↓
Token ID: [15496, 11, 527, 499, 345, 30]
↓
Embedding: [向量1, 向量2, 向量3, 向量4, 向量5, 向量6]

Token的重要性

- 1. 模型输入：模型只能处理数字，Token是文本到数字的桥梁
- 2. 词汇表大小：影响模型参数和计算量
- 3. 处理效率：合理的Token化可以提高处理效率
- 4. 多语言支持：好的Token化方法可以更好地处理多语言

特殊Token

- **<PAD>**：填充token，用于对齐序列长度
- **<UNK>**：未知token，表示词汇表中不存在的词
- **<BOS>** / **<SOS>**：序列开始token
- **<EOS>**：序列结束token
- **<MASK>**：掩码token（BERT中使用）
- **<SEP>**：分隔token，用于分隔不同句子

Token vs 字符 vs 词

特性	字符	Token（子词）	词
词汇表大小	很小（~100）	中等（~50k）	很大（>100k）
序列长度	很长	中等	较短
语义捕获	弱	强	强
未登录词	无问题	可处理	有问题
计算效率	低	高	中等

8. Agent与Workflow的区别?

Agent（智能体）

定义

Agent是一个自主的AI系统，能够感知环境、做出决策并执行行动，通常具有目标导向的行为。

特点

- **自主性**：能够独立做出决策
- **目标导向**：有明确的目标或任务
- **工具使用**：可以调用外部工具和API
- **推理能力**：能够进行多步推理和规划
- **适应性**：根据环境反馈调整行为

工作方式

用户输入 → Agent理解 → 规划步骤 → 执行行动 → 观察结果 → 调整策略 → 完成任务

示例

- **ReAct Agent**：结合推理（Reasoning）和行动（Acting）
- **AutoGPT**：自主完成复杂任务
- **LangChain Agent**：使用工具链解决问题

Workflow（工作流）

定义

Workflow是一个预定义的、结构化的执行流程，按照固定的步骤顺序执行任务。

特点

- **预定义性**：流程是预先设计好的
- **顺序性**：按照固定顺序执行步骤
- **确定性**：给定相同输入，产生相同输出
- **可预测性**：执行路径是已知的
- **可控性**：每个步骤都是明确的

工作方式



示例

- **数据处理流水线**：数据清洗 → 特征提取 → 模型训练
- **文档处理流程**：读取 → 解析 → 提取 → 存储
- **API调用链**：认证 → 查询 → 处理 → 返回

主要区别

特性	Agent	Workflow
决策方式	动态决策	预定义流程
灵活性	高，可适应变化	低，固定流程
自主性	高，自主选择行动	低，按步骤执行
复杂度	高，需要推理	低，线性执行
适用场景	复杂、不确定任务	标准化、重复性任务
错误处理	动态调整	预定义处理
可解释性	相对较低	高，流程清晰

使用场景

Agent适用于：

- 需要复杂推理的任务
- 目标明确但路径不确定
- 需要工具调用和外部交互
- 探索性任务
- 需要适应环境变化

Workflow适用于：

- 标准化业务流程
- 重复性任务

- 需要可预测性的场景
- 简单的线性处理
- 批处理任务

结合使用

在实际应用中，Agent和Workflow可以结合使用：

- **Workflow作为Agent的工具**：Agent可以调用预定义的Workflow
- **Agent管理Workflow**：Agent决定使用哪个Workflow
- **混合模式**：部分步骤用Workflow，部分用Agent决策

示例对比

Agent示例（任务：写一篇关于AI的博客）

1. Agent分析任务需求
2. Agent决定需要：研究资料、大纲、写作、校对
3. Agent搜索相关资料（调用搜索工具）
4. Agent根据资料生成大纲（调用LLM）
5. Agent撰写文章（调用LLM）
6. Agent检查文章质量（调用评估工具）
7. 如果质量不够，Agent返回步骤4重新生成

Workflow示例（任务：处理用户订单）

1. 接收订单（固定步骤）
2. 验证库存（固定步骤）
3. 处理支付（固定步骤）
4. 生成发货单（固定步骤）
5. 发送确认邮件（固定步骤）

9. Function Call是如何实现的？

Function Call的定义

Function Call（函数调用）是让LLM能够调用外部函数/工具的能力，使模型可以执行实际的操作，而不仅仅是生成文本。

实现原理

1. 工具定义 (Tool Definition)

首先需要定义可用的函数及其参数：

```
tools = [  
    {  
        "type": "function",  
        "function": {  
            "name": "get_weather",  
            "description": "获取指定城市的天气信息",  
            "parameters": {  
                "type": "object",  
                "properties": {  
                    "location": {  
                        "type": "string",  
                        "description": "城市名称, 例如: 北京、上海"  
                    },  
                    "unit": {  
                        "type": "string",  
                        "enum": ["celsius", "fahrenheit"],  
                        "description": "温度单位"  
                    }  
                },  
                "required": ["location"]  
            }  
        }  
    }  
]
```

2. 模型调用 (Model Invocation)

将工具定义和用户消息一起发送给模型：

```
response = client.chat.completions.create(  
    model="gpt-4",  
    messages=[  
        {"role": "user", "content": "北京今天天气怎么样? "}   
    ],  
    tools=tools,  
    tool_choice="auto" # 让模型决定是否调用工具  
)
```

3. 模型响应 (Model Response)

模型会返回是否需要调用函数：

```
{
  "role": "assistant",
  "content": null,
  "tool_calls": [
    {
      "id": "call_abc123",
      "type": "function",
      "function": {
        "name": "get_weather",
        "arguments": "{\"location\": \"北京\", \"unit\": \"celsius\"}"
      }
    }
  ]
}
```

4. 执行函数 (Execute Function)

根据模型返回的函数调用信息，执行实际函数：

```
if response.choices[0].message.tool_calls:
    tool_call = response.choices[0].message.tool_calls[0]
    function_name = tool_call.function.name
    function_args = json.loads(tool_call.function.arguments)

    # 执行函数
    if function_name == "get_weather":
        result = get_weather(
            location=function_args["location"],
            unit=function_args.get("unit", "celsius")
        )
```

5. 返回结果 (Return Results)

将函数执行结果返回给模型，让模型生成最终回答：

```
messages.append({
    "role": "tool",
    "tool_call_id": tool_call.id,
    "name": function_name,
    "content": json.dumps(result)
})

# 再次调用模型，生成最终回答
final_response = client.chat.completions.create(
    model="gpt-4",
    messages=messages
)
```

完整流程示例

```

import openai
import json

# 1. 定义工具
def get_weather(location, unit="celsius"):
    # 模拟天气API调用
    return {"temperature": 25, "condition": "晴天", "unit": unit}

tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "获取天气信息",
        "parameters": {
            "type": "object",
            "properties": {
                "location": {"type": "string"},
                "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
            },
            "required": ["location"]
        }
    }
}]

# 2. 用户消息
messages = [{"role": "user", "content": "北京今天天气怎么样? "}]

# 3. 模型调用（可能返回函数调用）
response = openai.chat.completions.create(
    model="gpt-4",
    messages=messages,
    tools=tools
)

# 4. 检查是否需要调用函数
if response.choices[0].message.tool_calls:
    tool_call = response.choices[0].message.tool_calls[0]
    function_name = tool_call.function.name
    function_args = json.loads(tool_call.function.arguments)

# 5. 执行函数
function_result = get_weather(**function_args)

# 6. 将结果返回给模型
messages.append(response.choices[0].message)
messages.append({
    "role": "tool",

```

```
        "tool_call_id": tool_call.id,  
        "name": function_name,  
        "content": json.dumps(function_result)  
    })  
  
    # 7. 获取最终回答  
    final_response = openai.chat.completions.create(  
        model="gpt-4",  
        messages=messages  
    )  
    print(final_response.choices[0].message.content)
```

关键技术点

1. 结构化输出

- 模型需要生成符合JSON Schema的结构化输出
- 通过训练，模型学会了如何生成函数调用的格式

2. 工具选择

- 模型需要从多个工具中选择合适的工具
- 基于工具描述和用户需求进行匹配

3. 参数提取

- 从自然语言中提取函数参数
- 处理参数类型转换和验证

4. 多轮对话

- 支持多次函数调用
- 函数结果可以作为下一轮调用的输入

应用场景

1. **API调用**：调用外部API获取实时数据
2. **数据库查询**：执行数据库操作
3. **代码执行**：执行代码并返回结果
4. **文件操作**：读写文件
5. **计算任务**：执行复杂计算

优势

- **扩展性**：模型可以访问外部工具和数据
 - **实时性**：可以获取最新信息
 - **准确性**：通过工具获取准确数据，而非依赖模型记忆
 - **灵活性**：可以组合多个工具完成复杂任务
-

10. RAG系统的核心构成要素是什么？

RAG的定义

RAG (Retrieval-Augmented Generation, 检索增强生成) 是一种结合信息检索和文本生成的技术，通过从外部知识库检索相关信息来增强LLM的生成能力。

核心构成要素

1. 文档存储 (Document Store)

- **作用**：存储原始文档和知识库
- **类型**：
 - 向量数据库 (Vector Database)：如Pinecone、Weaviate、Milvus、Qdrant
 - 传统数据库：如PostgreSQL、MongoDB
 - 文件系统：本地文件、云存储
- **要求**：支持高效的相似度搜索

2. 文档处理 (Document Processing)

- **文档加载**：
 - PDF、Word、Markdown、HTML等格式
 - 网页内容、API数据
- **文档分割 (Chunking)**：
 - 将长文档分割成小块
 - 策略：固定大小、按段落、按句子、滑动窗口
- **元数据提取**：
 - 文档来源、创建时间、作者等

3. 向量化 (Embedding)

- **作用：**将文本转换为向量表示
- **模型选择：**
 - 通用模型：OpenAI text-embedding-ada-002、text-embedding-3
 - 领域模型：针对特定领域训练的模型
 - 多语言模型：支持多语言嵌入
- **向量维度：**通常512-1536维

4. 检索器 (Retriever)

- **作用：**从知识库中检索相关文档
- **检索方法：**
 - **密集检索 (Dense Retrieval)：**基于向量相似度
 - 余弦相似度
 - 点积相似度
 - 欧氏距离
 - **稀疏检索 (Sparse Retrieval)：**基于关键词匹配
 - BM25
 - TF-IDF
 - **混合检索 (Hybrid Retrieval)：**结合密集和稀疏检索
- **检索策略：**
 - Top-K检索：返回最相关的K个文档
 - 重排序 (Re-ranking)：对检索结果重新排序

5. 生成器 (Generator / LLM)

- **作用：**基于检索到的上下文生成回答
- **模型选择：**
 - GPT-4、GPT-3.5-turbo
 - Claude、Llama、Mistral等
- **提示工程：**
 - 设计包含检索上下文的提示模板
 - 明确指示模型使用检索到的信息

6. 提示模板 (Prompt Template)

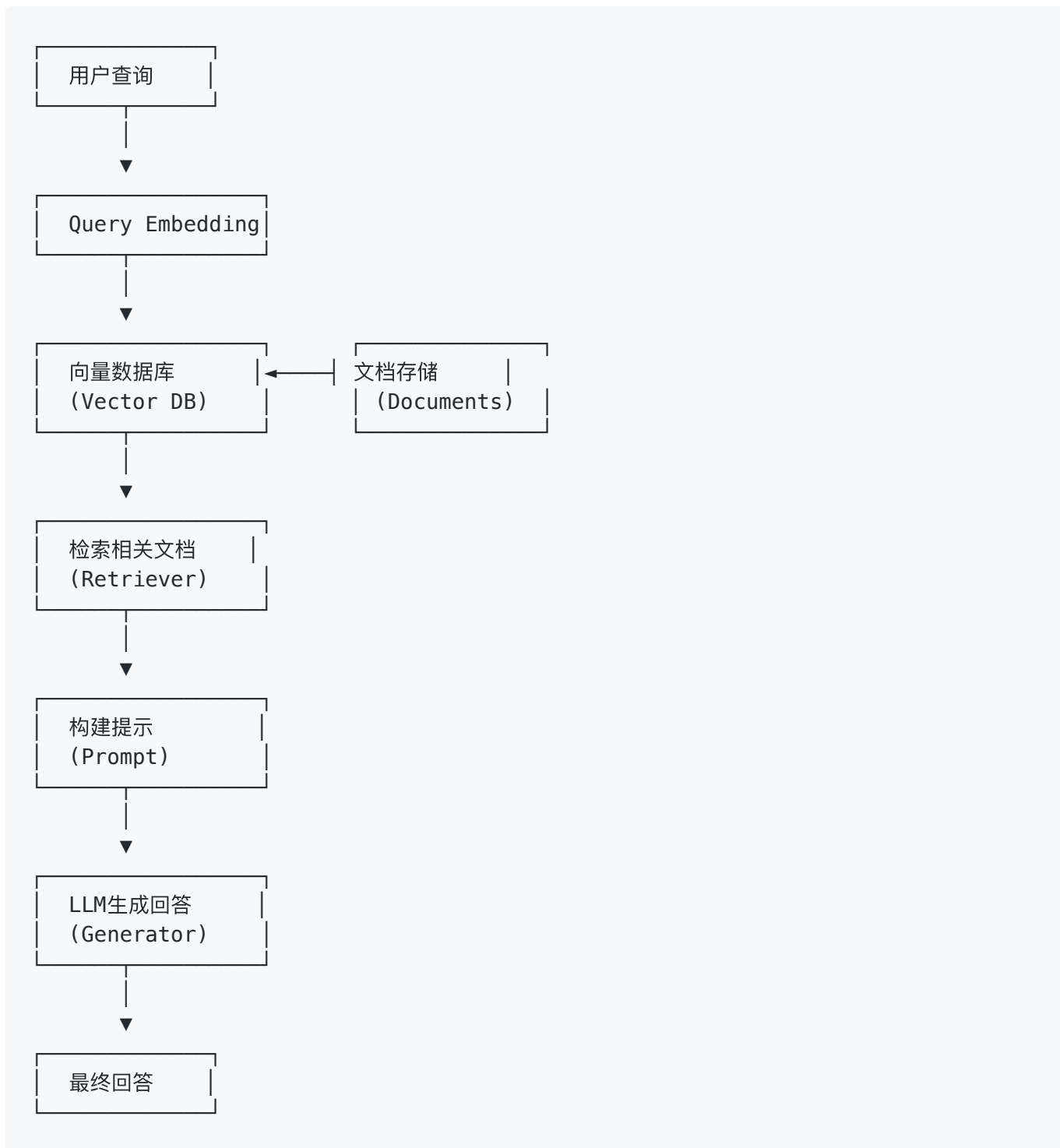
- **作用：**构建包含上下文和问题的提示
- **结构：**

系统提示：定义角色和任务
上下文：{检索到的文档}
用户问题：{用户查询}
指令：基于上下文回答问题

RAG工作流程

1. 用户查询
↓
2. 查询向量化 (Query Embedding)
↓
3. 向量相似度搜索 (Similarity Search)
↓
4. 检索相关文档块 (Retrieve Relevant Chunks)
↓
5. 构建提示 (Construct Prompt)
 - 系统提示
 - 检索到的上下文
 - 用户问题↓
6. LLM生成回答 (Generate Answer)
↓
7. 返回结果给用户

完整架构图



关键技术

1. 分块策略 (Chunking Strategy)

- 固定大小：每块固定token数（如512 tokens）
- 重叠窗口：块之间有一定重叠，避免信息丢失
- 语义分块：按段落或句子边界分割

2. 检索优化

- **重排序**：使用更强大的模型对检索结果重排序
- **多路检索**：同时使用多种检索方法
- **查询扩展**：扩展查询词，提高召回率

3. 上下文管理

- **上下文窗口限制**：管理LLM的上下文长度
- **相关性过滤**：过滤低相关性文档
- **去重**：去除重复的检索结果

优势

1. **知识更新**：可以访问最新信息，不依赖模型训练数据
2. **可追溯性**：可以追溯到信息来源
3. **准确性**：基于真实文档，减少幻觉
4. **领域适应**：可以快速适应新领域
5. **成本效益**：不需要重新训练模型

挑战

1. **检索质量**：检索不到相关信息会导致回答错误
2. **上下文长度**：检索到的文档可能超出模型上下文窗口
3. **文档质量**：知识库质量直接影响系统效果
4. **实时性**：需要平衡检索速度和准确性

11. 如何评价RAG系统的好与坏？

评价维度

1. 准确性 (Accuracy)

回答准确性

- **定义**：生成答案的正确性
- **评估方法**：
 - **人工评估**：专家判断答案是否正确
 - **自动评估**：与标准答案对比 (BLEU、ROUGE、BERTScore)
 - **事实检查**：验证答案中的事实是否正确

检索准确性

- 定义：检索到的文档是否与问题相关
- 评估指标：
 - 精确率 (**Precision**)：检索到的相关文档比例
 - 召回率 (**Recall**)：检索到的相关文档占所有相关文档的比例
 - **F1分数**：精确率和召回率的调和平均

2. 相关性 (Relevance)

- 定义：检索到的文档与查询的相关程度
- 评估方法：
 - **MRR (Mean Reciprocal Rank)**：第一个相关文档的排名倒数
 - 例如：第一个相关文档在第3位， $MRR = 1/3$
 - **NDCG (Normalized Discounted Cumulative Gain)**：考虑排名位置的相关性评分
 - **MAP (Mean Average Precision)**：平均精确率

3. 完整性 (Completeness)

- 定义：答案是否完整回答了问题
- 评估方法：
 - 检查是否回答了所有子问题
 - 是否提供了足够的细节
 - 是否遗漏了关键信息

4. 一致性 (Consistency)

- 定义：答案是否与检索到的文档一致
- 评估方法：
 - 检查是否存在矛盾
 - 验证答案是否基于检索到的上下文
 - 识别幻觉 (Hallucination)

5. 响应时间 (Latency)

- 定义：从查询到返回答案的时间
- 组成部分：
 - 查询向量化时间
 - 检索时间
 - LLM生成时间

- 目标：通常希望<3秒

6. 可追溯性 (Traceability)

- 定义：能否追溯到答案的来源
- 评估方法：
 - 是否提供引用来源
 - 引用是否准确
 - 是否易于验证

评估指标

1. 检索指标

```
# 精确率
Precision = (检索到的相关文档数) / (检索到的总文档数)

# 召回率
Recall = (检索到的相关文档数) / (所有相关文档数)

# F1分数
F1 = 2 * (Precision * Recall) / (Precision + Recall)

# MRR
MRR = (1 / 排名1 + 1 / 排名2 + ...) / 查询总数
```

2. 生成指标

- **BLEU**：基于n-gram匹配的机器翻译评估指标
- **ROUGE**：基于召回率的摘要评估指标
- **BERTScore**：基于语义相似度的评估
- **Faithfulness**：答案是否忠实于检索到的文档

3. 端到端指标

- **Answer Correctness**：答案正确性 (0-1分数)
- **Answer Relevance**：答案相关性
- **Context Precision**：上下文精确率
- **Context Recall**：上下文召回率

评估方法

1. 人工评估 (Human Evaluation)

- 优点：最准确，能理解语义和上下文
- 缺点：成本高，速度慢，主观性强
- 评估维度：
 - 准确性 (1-5分)
 - 相关性 (1-5分)
 - 完整性 (1-5分)
 - 可读性 (1-5分)

2. 自动评估 (Automatic Evaluation)

基于规则的评估

- 关键词匹配
- 实体识别和匹配
- 结构化数据验证

基于模型的评估

- 使用另一个LLM评估答案质量
- 使用专门的评估模型（如GPT-4作为评估器）

```
# 使用GPT-4评估答案
evaluation_prompt = f"""
请评估以下答案的质量（1-5分）：
问题：{question}
答案：{answer}
上下文：{context}

评估维度：
1. 准确性：答案是否正确
2. 相关性：答案是否相关
3. 完整性：答案是否完整
4. 一致性：答案是否与上下文一致
"""

score = gpt4_evaluator(evaluation_prompt)
```

3. 基准测试 (Benchmark)

常见基准数据集

- **Natural Questions**: 问答数据集
- **SQuAD**: 阅读理解数据集
- **MS MARCO**: 信息检索数据集
- **HotpotQA**: 需要多文档推理的问答

评估流程

```
# 1. 加载测试集
test_questions = load_benchmark("natural_questions")

# 2. 对每个问题运行RAG系统
results = []
for question in test_questions:
    answer = rag_system.query(question)
    results.append({
        "question": question,
        "predicted_answer": answer,
        "ground_truth": question["answer"]
    })

# 3. 计算指标
metrics = evaluate(results)
print(f"Accuracy: {metrics['accuracy']}")
print(f"F1: {metrics['f1']}")
```

评估框架

1. RAGAS (Retrieval-Augmented Generation Assessment)

专门用于评估RAG系统的框架:

```

from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy,
    context_precision,
    context_recall
)

# 评估数据
dataset = {
    "question": ["问题1", "问题2"],
    "answer": ["答案1", "答案2"],
    "contexts": [["上下文1"], ["上下文2"]],
    "ground_truth": ["标准答案1", "标准答案2"]
}

# 评估
results = evaluate(
    dataset=dataset,
    metrics=[
        faithfulness,
        answer_relevancy,
        context_precision,
        context_recall
    ]
)

```

2. TruLens

提供可解释的RAG评估：

```
from trulens_eval import Tru
from trulens_eval import Feedback, Huggingface

tru = Tru()

# 定义反馈函数
f_qa_relevance = Feedback(
    Huggingface().relevance,
    name="Answer Relevance"
).on_input_output()

# 评估
tru_rag = tru.RAG(
    rag,
    app_id="RAG",
    feedbacks=[f_qa_relevance]
)
```

评估检查清单

功能评估

- ☐ 能否正确回答简单问题？
- ☐ 能否回答复杂、多步骤问题？
- ☐ 能否处理模糊查询？
- ☐ 能否处理领域特定问题？

质量评估

- ☐ 答案是否准确？
- ☐ 答案是否完整？
- ☐ 答案是否相关？
- ☐ 是否出现幻觉？

性能评估

- ☐ 响应时间是否可接受？
- ☐ 能否处理并发请求？
- ☐ 系统是否稳定？

用户体验评估

- ☐ 答案是否易于理解？
- ☐ 是否提供来源引用？
- ☐ 错误处理是否友好？

持续改进

1. 收集反馈：用户反馈、错误日志
2. 分析失败案例：为什么某些查询失败？
3. 优化检索：改进检索策略和参数
4. 优化提示：改进提示模板
5. 更新知识库：添加缺失的文档

12. ChatGPT类应用的对话记忆是如何实现的？

对话记忆的重要性

对话记忆使AI能够：

- 记住之前的对话内容
- 理解上下文和指代关系
- 提供连贯的对话体验
- 维护对话状态

实现方式

1. 上下文窗口（Context Window）

基本实现

- 方式：将所有历史对话作为上下文发送给模型
- 结构：

```

messages = [
    {"role": "system", "content": "你是一个有用的助手"},
    {"role": "user", "content": "你好"},
    {"role": "assistant", "content": "你好! 有什么可以帮助你吗?"},
    {"role": "user", "content": "告诉我关于Python的信息"},
    {"role": "assistant", "content": "Python是一种编程语言..."},
    # ... 更多历史消息
    {"role": "user", "content": "刚才说的那个怎么用?"} # 当前问题
]

```

限制

- **Token限制**: 模型有最大上下文长度 (如GPT-4: 128k tokens)
- **成本**: 上下文越长, API调用成本越高
- **性能**: 上下文越长, 处理时间越长

2. 消息历史管理 (Message History Management)

滑动窗口 (Sliding Window)

- **策略**: 保留最近的N条消息
- **实现**:

```

def manage_history(messages, max_messages=20):
    if len(messages) > max_messages:
        # 保留系统消息和最近的对话
        system_msg = [msg for msg in messages if msg["role"] == "system"]
        recent_msgs = messages[-max_messages+1:] # +1 保留系统消息
        return system_msg + recent_msgs
    return messages

```

智能截断 (Smart Truncation)

- **策略**: 优先保留重要消息
- **方法**:
 - 保留系统提示
 - 保留最近的对话
 - 压缩或总结旧对话

3. 对话总结 (Conversation Summarization)

目的

- 压缩历史对话，节省上下文空间
- 保留关键信息

实现

```
def summarize_conversation(old_messages, current_messages):  
    # 将旧对话总结  
    old_text = format_messages(old_messages)  
    summary_prompt = f"请总结以下对话的关键信息: \n{old_text}"  
    summary = llm.generate(summary_prompt)  
  
    # 将总结作为系统消息的一部分  
    return [  
        {"role": "system", "content": f"之前的对话总结: {summary}"},  
        *current_messages  
    ]
```

4. 向量存储 (Vector Storage)

目的

- 将历史对话存储为向量
- 根据相关性检索历史信息

实现

```

from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings

# 初始化向量存储
vectorstore = Chroma(embedding_function=OpenAIEmbeddings())

# 存储对话
def store_conversation(messages, session_id):
    for msg in messages:
        if msg["role"] in ["user", "assistant"]:
            vectorstore.add_texts(
                texts=[msg["content"]],
                metadatas=[{
                    "session_id": session_id,
                    "role": msg["role"],
                    "timestamp": datetime.now()
                }]
            )

# 检索相关历史
def retrieve_relevant_history(query, session_id, top_k=3):
    results = vectorstore.similarity_search(
        query,
        filter={"session_id": session_id},
        k=top_k
    )
    return [r.page_content for r in results]

```

5. 外部记忆存储 (External Memory Storage)

数据库存储

使用数据库存储对话历史

```
import sqlite3
```

```
def save_message(session_id, role, content):
    conn = sqlite3.connect('conversations.db')
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO messages (session_id, role, content, timestamp) VALUES (?, ?, ?, ?)",
        (session_id, role, content, datetime.now())
    )
    conn.commit()
    conn.close()

def load_conversation(session_id, limit=50):
    conn = sqlite3.connect('conversations.db')
    cursor = conn.cursor()
    cursor.execute(
        "SELECT role, content FROM messages WHERE session_id = ? ORDER BY timestamp DESC LIMIT ?",
        (session_id, limit)
    )
    messages = [{"role": row[0], "content": row[1]} for row in cursor.fetchall()]
    conn.close()
    return list(reversed(messages)) # 按时间正序返回
```

Redis缓存

```
import redis
import json

redis_client = redis.Redis(host='localhost', port=6379, db=0)

def save_conversation(session_id, messages):
    redis_client.setex(
        f"conversation:{session_id}",
        3600, # 1小时过期
        json.dumps(messages)
    )

def load_conversation(session_id):
    data = redis_client.get(f"conversation:{session_id}")
    if data:
        return json.loads(data)
    return []
```

6. 会话管理 (Session Management)

会话标识

- 每个用户对话分配唯一会话ID
- 基于用户ID、设备ID或临时会话ID

实现

```
class ConversationManager:
    def __init__(self):
        self.sessions = {} # session_id -> messages

    def get_or_create_session(self, session_id):
        if session_id not in self.sessions:
            self.sessions[session_id] = []
        return self.sessions[session_id]

    def add_message(self, session_id, role, content):
        messages = self.get_or_create_session(session_id)
        messages.append({"role": role, "content": content})

        # 限制消息数量
        if len(messages) > 100:
            messages = messages[-100:]
            self.sessions[session_id] = messages

    def get_messages(self, session_id, limit=50):
        messages = self.get_or_create_session(session_id)
        return messages[-limit:] if len(messages) > limit else messages
```

完整实现示例

```

class ChatMemory:
    def __init__(self, max_context_tokens=8000, max_messages=50):
        self.max_context_tokens = max_context_tokens
        self.max_messages = max_messages
        self.sessions = {}

    def add_message(self, session_id, role, content):
        if session_id not in self.sessions:
            self.sessions[session_id] = []

        self.sessions[session_id].append({
            "role": role,
            "content": content,
            "timestamp": datetime.now()
        })

        # 管理消息数量
        self._trim_messages(session_id)

    def get_context(self, session_id, current_query):
        messages = self.sessions.get(session_id, [])

        # 构建完整上下文
        context_messages = [
            {"role": "system", "content": "你是一个有用的助手"}
        ]

        # 添加历史消息（考虑token限制）
        token_count = self._count_tokens(context_messages)
        for msg in reversed(messages[:-1]): # 不包括当前查询
            msg_tokens = self._count_tokens([msg])
            if token_count + msg_tokens > self.max_context_tokens:
                break
            context_messages.insert(1, msg)
            token_count += msg_tokens

        # 添加当前查询
        context_messages.append({
            "role": "user",
            "content": current_query
        })

        return context_messages

    def _trim_messages(self, session_id):
        messages = self.sessions[session_id]
        if len(messages) > self.max_messages:

```



```
# 保留系统消息和最近的对话
self.sessions[session_id] = messages[-self.max_messages:]

def _count_tokens(self, messages):
    # 简化的token计数 (实际应使用tokenizer)
    text = " ".join([msg["content"] for msg in messages])
    return len(text.split()) * 1.3 # 粗略估计
```

高级技术

1. 记忆压缩 (Memory Compression)

- 定期总结旧对话
- 只保留关键信息

2. 分层记忆 (Hierarchical Memory)

- 短期记忆：当前对话
- 中期记忆：最近几轮对话
- 长期记忆：总结和关键信息

3. 语义检索记忆 (Semantic Memory Retrieval)

- 基于当前查询检索相关历史
- 而不是简单地使用最近的消息

最佳实践

1. 平衡上下文长度：在记忆完整性和性能之间平衡
2. 智能截断：优先保留重要信息
3. 定期清理：删除过期或无关的对话
4. 隐私保护：安全存储用户对话数据
5. 性能优化：使用缓存和异步处理

13. 什么是MCP (Model Context Protocol) ?

MCP的定义

Model Context Protocol (MCP) 是由Anthropic在2024年11月开发的一个开源标准，旨在促进人工智能系统（特别是大语言模型）与外部工具、服务和数据源之间的无缝集成。

核心特性

1. 标准化集成 (Standardized Integration)

- 提供统一的框架连接AI模型与外部系统
- 减少自定义集成的需求
- 简化开发过程

2. 客户端-服务器架构 (Client-Server Architecture)

- **客户端 (Client)**：AI应用程序（如ChatGPT桌面应用）
- **服务器 (Server)**：提供资源、工具和预定义查询的服务
- **通信方式**：安全的双向通信

3. 多语言支持 (Multi-Language Support)

- 提供多种编程语言的SDK：
 - Python
 - TypeScript
 - C#
 - Java
- 支持跨平台开发

主要功能

1. 资源访问 (Resources)

- 提供对数据、文档、知识库的访问
- 支持实时数据获取
- 示例：访问数据库、文件系统、API

2. 工具调用 (Tools)

- 暴露可执行的函数和操作
- AI模型可以调用这些工具
- 示例：执行代码、发送邮件、操作文件

3. 提示模板 (Prompts)

- 预定义的查询模板
- 可重用的提示结构
- 提高一致性和效率

应用场景

1. 软件开发

- **IDE集成**：如Zed编辑器
- **代码平台**：如Replit
- **代码智能工具**：如Sourcegraph
- **功能**：为AI编程助手提供实时项目上下文

2. 企业解决方案

- **内部工具集成**：如Block公司
- **数据访问**：从专有文档、CRM系统、知识库检索数据
- **决策支持**：改善决策过程

3. 学术研究

- **参考文献管理**：如Zotero集成
- **研究工具**：语义搜索、注释提取、文献综述生成

行业采用

主要支持者

1. OpenAI (2025年3月)

- ChatGPT桌面应用
- Agents SDK
- 全面采用MCP

2. Google DeepMind

- 在即将发布的Gemini模型中支持MCP
- 相关基础设施集成

3. Microsoft

- GitHub集成

- Microsoft 365集成
- Azure平台支持

4. Anthropic (2024年12月)

- 将MCP捐赠给Agentic AI Foundation (AAIF)
- 确保开源和社区驱动
- 在Linux Foundation下管理

技术架构

通信协议

```
AI应用 (客户端)
  ↓ JSON-RPC over stdio/HTTP/SSE
MCP服务器
  ↓
外部系统 (数据库、API、文件系统等)
```

消息格式

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "location": "北京"
    }
  }
}
```

安全考虑

潜在风险

1. 身份碎片化 (Identity Fragmentation)

- MCP服务器可能成为攻击目标
- 需要统一身份管理

2. 提示注入 (Prompt Injection)

- 恶意输入可能影响AI行为
- 需要输入验证和清理

安全最佳实践

1. 身份验证：实施强大的身份验证机制
2. 加密认证：使用加密认证替代静态凭证
3. 统一身份管理：集成到统一的身份和访问管理框架
4. 权限控制：限制MCP服务器的访问权限
5. 输入验证：验证和清理所有输入

MCP的优势

1. 互操作性：标准化的接口，易于集成
2. 扩展性：可以轻松添加新的工具和资源
3. 灵活性：支持多种通信方式（stdio、HTTP、SSE）
4. 社区驱动：开源标准，社区参与
5. 跨平台：支持多种编程语言和平台

使用示例

MCP服务器示例 (Python)

```

from mcp.server import Server
from mcp.types import Tool, Resource

app = Server("weather-server")

@app.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(
            name="get_weather",
            description="获取指定城市的天气信息",
            inputSchema={
                "type": "object",
                "properties": {
                    "location": {"type": "string"}
                },
                "required": ["location"]
            }
        )
    ]

@app.call_tool()
async def call_tool(name: str, arguments: dict):
    if name == "get_weather":
        location = arguments.get("location")
        # 调用天气API
        weather_data = fetch_weather(location)
        return {"content": weather_data}

```

MCP客户端示例

```

from mcp import ClientSession, StdioServerParameters
import asyncio

async def main():
    # 连接到MCP服务器
    server_params = StdioServerParameters(
        command="python",
        args=["weather_server.py"]
    )

    async with ClientSession(server_params) as session:
        # 列出可用工具
        tools = await session.list_tools()
        print(f"可用工具: {[t.name for t in tools.tools]}")

        # 调用工具
        result = await session.call_tool(
            "get_weather",
            {"location": "北京"}
        )
        print(f"天气信息: {result.content}")

asyncio.run(main())

```

未来展望

1. **标准化**: 成为AI集成的行业标准
2. **生态系统**: 丰富的MCP服务器和工具生态
3. **安全性**: 持续改进安全机制
4. **性能**: 优化通信和性能
5. **社区**: 在Linux Foundation下持续发展

总结

MCP是一个重要的AI集成标准，它：

- 提供了标准化的方式连接AI和外部系统
 - 得到了主要AI公司的支持
 - 正在成为AI应用开发的重要基础设施
 - 通过开源和社区驱动确保长期发展
-

参考资料

- [LLMs from Scratch](#)
 - [Model Context Protocol](#)
 - [RAG Evaluation](#)
 - [LangChain Documentation](#)
-

本文档持续更新中，如有问题或建议，欢迎反馈。